COMPUTER VISION

ALL PRACTICALS – UNIT 1 & UNIT 2

Beginner-friendly Python programs with aims, code, inputs and setup notes

SECTION	CONTENTS
Unit 1	10 practicals (including all four parts of Practical 4)
Unit 2	12 mini-project practicals
Total	22 numbered practicals / exercises

Before you start

Install the required packages from the setup page below. Keep input images, videos, datasets and pretrained model files in the paths mentioned under each practical. File names and folder names are case-sensitive on some systems.

General package installation

```
pip install opencv-python opencv-contrib-python numpy matplotlib tensorflow scikit-learn ultralytics pytesseract rembg pillow open3d trans
```

Some practicals also require separate software, datasets or pretrained weights. Those requirements are stated with the relevant practical.

UNIT 1 – COMPUTER VISION PRACTICALS

Practical 1. Perform Geometric Transformation

Aim: To perform resizing, rotation, translation and flipping on an image.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Resize
resize = cv2.resize(img, (300, 300))

# Rotation
rows, cols = img.shape[:2]
matrix = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)
rotate = cv2.warpAffine(img, matrix, (cols, rows))

# Translation
matrix = cv2.getRotationMatrix2D((cols/2, rows/2), 0, 1)
matrix[0, 2] = 50
matrix[1, 2] = 30
translate = cv2.warpAffine(img, matrix, (cols, rows))

# Flip
flip = cv2.flip(img, 1)

plt.figure(figsize=(10, 8))
plt.subplot(2, 2, 1); plt.imshow(resize); plt.title("Resize")
plt.subplot(2, 2, 2); plt.imshow(rotate); plt.title("Rotation")
plt.subplot(2, 2, 3); plt.imshow(translate); plt.title("Translation")
plt.subplot(2, 2, 4); plt.imshow(flip); plt.title("Flipping")
plt.show()
```

Input / Setup: Input: image.jpg.

Practical 2. Perform Image Stitching

Aim: To combine two images into a single panoramic image.

```
import cv2

img1 = cv2.imread("image1.jpg")
img2 = cv2.imread("image2.jpg")

stitcher = cv2.Stitcher_create()
status, result = stitcher.stitch([img1, img2])

if status == cv2.Stitcher_OK:
    cv2.imwrite("stitched_image.jpg", result)
    cv2.imshow("Stitched Image", result)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Image stitching failed")
```

Input / Setup: Input: image1.jpg and image2.jpg. Images should overlap sufficiently.

Practical 3. Perform Camera Calibration

Aim: To calibrate a camera using multiple chessboard images.

```
import cv2
import numpy as np
import glob

objpoints = []
imgpoints = []

objp = np.zeros((6 * 7, 3), np.float32)
objp[:, :2] = np.mgrid[0:7, 0:6].T.reshape(-1, 2)

images = glob.glob("calibration/*.jpg")

for image in images:
    img = cv2.imread(image)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    ret, corners = cv2.findChessboardCorners(gray, (7, 6), None)

    if ret:
        objpoints.append(objp)
```

```
        imgpoints.append(corners)

if len(objpoints) > 0:
    ret, matrix, distortion, rvecs, tvecs = cv2.calibrateCamera(
        objpoints, imgpoints, gray.shape[::-1], None, None
    )
    print("Camera Matrix:")
    print(matrix)
    print("Distortion Coefficients:")
    print(distortion)
else:
    print("No chessboard corners found. Check calibration images.")
```

Input / Setup: Input: calibration/ folder containing several chessboard photos. Pattern size (7, 6) means 7 by 6 inner corners.

Practical 4(i). Face Detection

Aim: To detect faces in an image using Haar Cascade.

```
import cv2

img = cv2.imread("image.jpg")
face = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face.detectMultiScale(gray, 1.3, 5)

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

cv2.imshow("Face Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Input: image.jpg.

Practical 4(ii). Object Detection

Aim: To detect common objects using a pretrained YOLO model.

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
img = cv2.imread("image.jpg")

results = model(img)
output = results[0].plot()

cv2.imshow("Object Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Input: image.jpg. The YOLO weights may download automatically on first run.

Practical 4(iii). Pedestrian Detection

Aim: To detect pedestrians using the HOG descriptor.

```
import cv2

img = cv2.imread("image.jpg")
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

boxes, weights = hog.detectMultiScale(img, winStride=(8, 8))

for (x, y, w, h) in boxes:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)

cv2.imshow("Pedestrian Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Input: image.jpg.

Practical 4(iv). Face Recognition

Aim: To train an LBPH face recognizer and recognize a test face.

```
import cv2
import os
import numpy as np
```

```
faces = []
labels = []
names = []

# Folder structure: faces/PersonName/image1.jpg
for label, person in enumerate(os.listdir("faces")):
    person_path = os.path.join("faces", person)
    if not os.path.isdir(person_path):
        continue
    names.append(person)

    for file in os.listdir(person_path):
        path = os.path.join(person_path, file)
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            continue
        img = cv2.resize(img, (100, 100))
        faces.append(img)
        labels.append(label)

recognizer = cv2.face.LBPHFaceRecognizer_create()
recognizer.train(np.array(faces), np.array(labels))
recognizer.save("face_model.yml")
print("Face model trained and saved.")

# Recognition
test = cv2.imread("test.jpg", cv2.IMREAD_GRAYSCALE)
test = cv2.resize(test, (100, 100))
label, confidence = recognizer.predict(test)
print("Person:", names[label])
print("LBPH distance (lower is generally a closer match):", confidence)
```

Input / Setup: Install opencv-contrib-python. Create faces/PersonName/ folders with several face images per person. The confidence value is an LBPH distance, not a probability.

Practical 5. Construct 3D Model from Images

Aim: To create and visualize a basic 3D point cloud. This is a visualization demonstration, not full photo-based reconstruction.

```
import open3d as o3d
import numpy as np

# Demonstration 3D points
points = np.random.rand(1000, 3)

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(points)

o3d.visualization.draw_geometries([pcd])
```

Input / Setup: For actual 3D reconstruction from photographs, use photogrammetry software such as COLMAP with multiple overlapping images and camera-pose estimation.

Practical 6. Object Detection and Tracking from Video

Aim: To detect and track objects in a video using YOLO.

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture("video.mp4")

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    results = model.track(frame, persist=True)
    output = results[0].plot()
    cv2.imshow("Object Tracking", output)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Input / Setup: Input: video.mp4. Press q to stop.

Practical 7. Feature Extraction using RANSAC

Aim: To extract SIFT features, match them and estimate a homography using RANSAC.

```
import cv2
import numpy as np

img1 = cv2.imread("image1.jpg")
img2 = cv2.imread("image2.jpg")

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)

good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append(m)

if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    print("Homography Matrix:")
    print(H)
else:
    print("Not enough matching features")

result = cv2.drawMatches(img1, kp1, img2, kp2, good, None)
cv2.imshow("Feature Matching", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Input: image1.jpg and image2.jpg. Images should contain common visual features.

Practical 8. Perform Colorization

Aim: To colorize a grayscale image using a pretrained deep-learning colorization model.

```
import cv2
import numpy as np

net = cv2.dnn.readNetFromCaffe(
    "colorization_deploy_v2.prototxt",
    "colorization_release_v2.caffemodel"
)

pts = np.load("pts_in_hull.npy")
class8 = net.getLayerId("class8_ab")
conv8 = net.getLayerId("conv8_313_rh")

net.getLayer(class8).blobs = [
    pts.transpose().reshape(2, 313, 1, 1)
]
net.getLayer(conv8).blobs = [
    np.full((1, 313), 2.606, dtype="float32")
]

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_rgb = cv2.cvtColor(gray, cv2.COLOR_GRAY2RGB)

scaled = gray_rgb.astype("float32") / 255.0
lab = cv2.cvtColor(scaled, cv2.COLOR_RGB2Lab)
L = lab[:, :, 0]

net.setInput(cv2.dnn.blobFromImage(L, 1.0, (224, 224), (50,)))
ab = net.forward()[0].transpose(1, 2, 0)
ab = cv2.resize(ab, (img.shape[1], img.shape[0]))

lab_output = np.concatenate((L[:, :, np.newaxis], ab), axis=2)
output = cv2.cvtColor(lab_output, cv2.COLOR_Lab2BGR)

cv2.imshow("Colorized Image", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Required model files: colorization_deploy_v2.prototxt, colorization_release_v2.caffemodel and pts_in_hull.npy.

Practical 9. Text Detection and Recognition

Aim: To recognize text from an image using Tesseract OCR.

```
import cv2
import pytesseract
```

```
# On Windows, set the executable path if Tesseract is not in PATH:
# pytesseract.pytesseract.tesseract_cmd = (
#     r"C:\Program Files\Tesseract-OCR\tesseract.exe"
# )

img = cv2.imread("text.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = cv2.threshold(
    gray, 150, 255, cv2.THRESH_BINARY
)[1]

text = pytesseract.image_to_string(gray)

print("Recognized Text:")
print(text)

cv2.imshow("Text Image", gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Install Tesseract OCR software separately, in addition to pip install pytesseract. Input: text.jpg.

Practical 10. Image Matting and Compositing

Aim: To remove an image background and place the foreground on a new background.

```
from rembg import remove
from PIL import Image

img = Image.open("person.jpg")
foreground = remove(img)
foreground.save("transparent.png")

background = Image.open("background.jpg")
background = background.resize(foreground.size)

background.paste(foreground, (0, 0), foreground)
background.save("final_image.jpg")

print("Image compositing completed.")
```

Input / Setup: Inputs: person.jpg and background.jpg. The rembg package may download model weights on first run.

UNIT 2 – MINI PROJECT PRACTICALS

Practical 1. Traffic Sign Classification using a Standard Benchmark Dataset

Aim: To classify traffic-sign images using a CNN and the GTSRB dataset.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

train = keras.utils.image_dataset_from_directory(
    "TrafficSigns/Train",
    image_size=(32, 32),
    batch_size=32,
    label_mode="int"
)

num_classes = len(train.class_names)

model = keras.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(train, epochs=5)
model.save("traffic_sign_model.keras")
print("Traffic sign classification completed.")
```

Input / Setup: Dataset: GTSRB. Folder example: TrafficSigns/Train/class_name/image files. Keep the same class-folder structure for all classes.

Practical 2. Multi-Class Plant Disease Classification

Aim: To classify plant diseases from leaf images using a CNN and PlantVillage dataset.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

train = keras.utils.image_dataset_from_directory(
    "PlantVillage",
    image_size=(128, 128),
    batch_size=32,
    label_mode="int"
)

num_classes = len(train.class_names)

model = keras.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(train, epochs=5)
model.save("plant_disease_model.keras")
print("Plant disease classification completed.")
```

Input / Setup: Dataset: PlantVillage. Each disease/healthy category must be a separate subfolder.

Practical 3. OCR for Handwritten Text

Aim: To recognize handwritten digits using the MNIST benchmark dataset.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

model = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    x_train, y_train,
    epochs=5,
    validation_data=(x_test, y_test)
)

prediction = model.predict(x_test[:1])
print("Predicted digit:", prediction.argmax())

plt.imshow(x_test[0], cmap="gray")
plt.show()
```

Input / Setup: This simple MNIST implementation recognizes handwritten digits (0–9), not complete handwritten words or sentences.

Practical 4. Facial Emotion Recognition

Aim: To classify facial expressions using a CNN and FER-2013 dataset.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

train = keras.utils.image_dataset_from_directory(
    "FER/train",
    image_size=(48, 48),
    color_mode="grayscale",
    batch_size=32
)

test = keras.utils.image_dataset_from_directory(
    "FER/test",
    image_size=(48, 48),
    color_mode="grayscale",
    batch_size=32
)

num_classes = len(train.class_names)

model = keras.Sequential([
    layers.Input(shape=(48, 48, 1)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(train, epochs=5, validation_data=test)
model.save("emotion_model.keras")
```

```
print("Facial emotion recognition completed.")
```

Input / Setup: Dataset: FER-2013. Use matching class folders in train and test (commonly angry, disgust, fear, happy, sad, surprise, neutral).

Practical 5. Food Image Classification or Clothing Classifier

Aim: To classify clothing images using the Fashion-MNIST dataset.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

names = [
    "T-shirt", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle Boot"
]

model = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(x_train, y_train, epochs=5)

prediction = model.predict(x_test[:1])
print("Predicted clothing:", names[prediction.argmax()])

plt.imshow(x_test[0], cmap="gray")
plt.show()
```

Input / Setup: This implements the clothing-classifier option using Fashion-MNIST.

Practical 6. Multi-Object Tracking in Video

Aim: To detect and track multiple objects in a video using YOLO and ByteTrack.

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture("video.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model.track(
        frame,
        persist=True,
        tracker="bytetrack.yaml"
    )

    output = results[0].plot()
    cv2.imshow("Multi Object Tracking", output)

    if cv2.waitKey(1) == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Input / Setup: Input: video.mp4. Press q to stop.

Practical 7. Image Captioning

Aim: To generate a natural-language caption for an image using a pretrained BLIP model.

```
from transformers import pipeline
```

```
from PIL import Image

captioner = pipeline(
    "image-to-text",
    model="Salesforce/blip-image-captioning-base"
)

img = Image.open("image.jpg").convert("RGB")
result = captioner(img)

print("Image caption:")
print(result[0]["generated_text"])
```

Input / Setup: Install transformers, torch and pillow. Input: image.jpg. The pretrained model may download on first run.

Practical 8. Gesture Recognition for Human-Computer Interaction

Aim: To classify hand-gesture images using a CNN.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

train = keras.utils.image_dataset_from_directory(
    "Gestures/Train",
    image_size=(64, 64),
    batch_size=32
)

num_classes = len(train.class_names)

model = keras.Sequential([
    layers.Input(shape=(64, 64, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(train, epochs=5)
model.save("gesture_model.keras")
print("Gesture recognition completed.")
```

Input / Setup: Input folder: Gestures/Train/class_name/image files. Each gesture must have its own folder.

Practical 9. Visual Question Answering (VQA)

Aim: To answer a question about the contents of an image using a pretrained BLIP VQA model.

```
from transformers import pipeline
from PIL import Image

vqa = pipeline(
    "visual-question-answering",
    model="Salesforce/blip-vqa-base"
)

img = Image.open("image.jpg").convert("RGB")
question = "What is in the image?"

result = vqa(image=img, question=question)

print("Question:", question)
print("Answer:", result[0]["answer"])
```

Input / Setup: Install transformers, torch and pillow. Input: image.jpg.

Practical 10. Face De-Aging / Aging – Age Estimation Demonstration

Aim: To estimate an approximate age range from a detected face using pretrained OpenCV DNN models. This is age estimation, not image aging/de-aging.

```
import cv2
import numpy as np
```

```
face_net = cv2.dnn.readNetFromCaffe(
    "deploy.prototxt",
    "res10_300x300_ssd_iter_140000.caffemodel"
)

age_net = cv2.dnn.readNetFromCaffe(
    "age_deploy.prototxt",
    "age_net.caffemodel"
)

age_list = [
    "(0-2)", "(4-6)", "(8-12)", "(15-20)",
    "(25-32)", "(38-43)", "(48-53)", "(60-100)"
]

img = cv2.imread("face.jpg")
h, w = img.shape[:2]

blob = cv2.dnn.blobFromImage(
    img, 1.0, (300, 300), [104, 177, 123]
)
face_net.setInput(blob)
detections = face_net.forward()

for i in range(detections.shape[2]):
    confidence = detections[0, 0, i, 2]

    if confidence > 0.5:
        box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
        x1, y1, x2, y2 = box.astype(int)

        x1, y1 = max(0, x1), max(0, y1)
        x2, y2 = min(w, x2), min(h, y2)
        face = img[y1:y2, x1:x2]

        if face.size == 0:
            continue

        blob = cv2.dnn.blobFromImage(
            face, 1.0, (227, 227),
            [78.426, 87.768, 114.895],
            swapRB=False
        )
        age_net.setInput(blob)
        prediction = age_net.forward()
        age = age_list[prediction[0].argmax()]

        cv2.putText(
            img, age, (x1, max(20, y1-10)),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2
        )
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

cv2.imshow("Age Estimation", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Required files: deploy.prototxt, res10_300x300_ssd_iter_140000.caffemodel, age_deploy.prototxt and age_net.caffemodel. For actual face aging/de-aging, use a generative image-to-image model.

Practical 11. Human Pose Estimation and Action Recognition in Crowded Scenes

Aim: To estimate the poses of multiple people in a video using a pretrained YOLO pose model.

```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n-pose.pt")
cap = cv2.VideoCapture("video.mp4")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame)
    output = results[0].plot()

    cv2.imshow("Human Pose Estimation", output)

    if cv2.waitKey(1) == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Input / Setup: This code performs multi-person pose estimation and draws keypoints/skeletons. Full action recognition (walking, running, jumping) requires an additional temporal action-classification model.

Practical 12. Unsupervised Anomaly Detection in Industrial Inspection

Aim: To identify abnormal images using an Isolation Forest trained on normal image examples.

```
import cv2
import numpy as np
import glob
from sklearn.ensemble import IsolationForest

images = glob.glob("normal/*.jpg")
features = []

for image in images:
    img = cv2.imread(image)
    if img is None:
        continue

    img = cv2.resize(img, (64, 64))
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    features.append(gray.flatten())

if len(features) == 0:
    raise ValueError("No valid training images found in normal/")

model = IsolationForest(
    contamination=0.1,
    random_state=42
)
model.fit(features)

test = cv2.imread("test.jpg")
if test is None:
    raise ValueError("Could not read test.jpg")

test = cv2.resize(test, (64, 64))
gray = cv2.cvtColor(test, cv2.COLOR_BGR2GRAY)
test_feature = gray.flatten().reshape(1, -1)

prediction = model.predict(test_feature)

if prediction[0] == 1:
    print("Normal Image")
else:
    print("Anomaly Detected")

cv2.imshow("Test Image", test)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input / Setup: Place only normal training images in normal/. Provide the inspection image as test.jpg. This is a simple baseline; industrial-quality inspection usually needs stronger feature representations.

Quick Practical Checklist

Unit 1

1. Geometric Transformation
2. Image Stitching
3. Camera Calibration
4. Face Detection, Object Detection, Pedestrian Detection and Face Recognition
5. 3D Model Construction
6. Object Detection and Tracking from Video
7. Feature Extraction using RANSAC
8. Colorization
9. Text Detection and Recognition
10. Image Matting and Compositing

Unit 2

1. Traffic Sign Classification
2. Plant Disease Classification
3. Handwritten OCR
4. Facial Emotion Recognition
5. Food Image Classification or Clothing Classifier
6. Multi-Object Tracking
7. Image Captioning
8. Gesture Recognition
9. Visual Question Answering
10. Face Aging / De-Aging (age-estimation demonstration)
11. Human Pose Estimation and Action Recognition
12. Unsupervised Industrial Anomaly Detection

Important academic notes

These are educational starter implementations. A few titles describe broader tasks than the short demonstration code fully implements: Practical 5 in Unit 1 visualizes a sample 3D point cloud; Practical 10 in Unit 2 estimates an age range rather than generating an aged/de-aged face; Practical 11 in Unit 2 estimates poses but does not classify actions. For a full mini-project submission, include a train/test split, evaluation metrics, screenshots, results and a presentation.